COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/ Data Structures and Algorithms	**SECTION:**	CPE2124
DATE PERFORMED:	Jul 31, 2026	**DATE SUBMITTED:**	Jul 31, 2026
NAME:	Condesa, Andrie G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4 **[STACKS]**

1. PROCEDURE OUTPUT

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROCEDURE

ILOA

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main()
{
    // Calling Stack from the STL
    stack<int> newStack;

    // Adds elements to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    // Returns true (1) if the stack is empty, otherwise false (0)
    cout << "Stack Empty? " << newStack.empty() << endl;

    // Returns the size of the stack
    cout << "Stack Size: " << newStack.size() << endl;

    // Returns the topmost element of the stack
    cout << "Top Element of the Stack: " << newStack.top() << endl;

    // Removes the topmost element
    newStack.pop();

```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

din=Microsoft-MIEngine-In-fpvervdh.bw2' '--stdout=Microsoft-MIEngine-Out-5miz2laf.a1i' '--stderr=Microsoft-MIEn
--interpreter=mi'
Stack Empty? 0
Stack Size: 3
Top Element of the Stack: 15
Top Element of the Stack: 8
Stack Size: 2
PS C:\Users\TIPQC\Documents\CONDESA>
```

ILOB.1

```cpp
#include <iostream>
using namespace std;

const int maxCap = 100;

int stack[maxCap];
int top = -1;
int limit;

void push();
void pop();
void Top();
void display();
bool isEmpty();

int main() {
    int choice;

    cout << "Enter maximum number of elements: ";
    cin >> limit;

    do {
        cout << "\n--- Stack Menu ---" << endl;
        cout << "1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Top" << endl;
        cout << "4. Is Empty" << endl;
        cout << "5. Display" << endl;
        cout << "6. Exit" << endl;
        cout << "Choice: ";
        cin >> choice;

```

```
32
33          switch (choice) {
34              case 1:
35                  push();
36                  break;
37              case 2:
38                  pop();
39                  break;
40              case 3:
41                  Top();
42                  break;
43              case 4:
44                  if (isEmpty())
45                      cout << "Stack is Empty" << endl;
46                  else
47                      cout << "Stack is Not Empty" << endl;
48                  break;
49              case 5:
50                  display();
51                  break;
52              case 6:
53                  cout << "Program Ended." << endl;
54                  break;
55              default:
56                  cout << "Invalid Choice!" << endl;
57          }
58
59      } while (choice != 6);
60
61      return 0;
```

```
60
61      return 0;
62  }
63
64  bool isEmpty() {
65      return (top == -1);
66  }
67
68  void push() {
69      if (top == limit - 1) {
70          cout << "Stack Overflow!" << endl;
71          return;
72      }
73
74      int value;
75      cout << "Enter value: ";
76      cin >> value;
77
78      top++;
79      stack[top] = value;
80  }
81
82  void pop() {
83      if (isEmpty()) {
84          cout << "Stack Underflow!" << endl;
85          return;
86      }
87
88      cout << "Removed: " << stack[top] << endl;
89      top--;
90  }
```

```
82  void pop() {
83      if (isEmpty()) {
87
88      cout << "Removed: " << stack[top] << endl;
89      top--;
90  }
91
92  void Top() {
93      if (isEmpty()) {
94          cout << "Stack is Empty!" << endl;
95      } else {
96          cout << "Top Element: " << stack[top] << endl;
97      }
98  }
99
100 void display() {
101     if (isEmpty()) {
102         cout << "Stack is Empty!" << endl;
103         return;
104     }
105
106     cout << "Stack Elements (Top to Bottom):" << endl;
107     for (int i = top; i >= 0; i--) {
108         cout << stack[i] << endl;
109     }
110 }
111
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
din=Microsoft-MIEngine-In-fpvervdh.bw2' '--stdout=Microsoft-MIEngine-Out-5miz2laf.a1i' '--std
--interpreter=mi'
Stack Empty? 0
Stack Size: 3
Top Element of the Stack: 15
Top Element of the Stack: 8
Stack Size: 2
PS C:\Users\TIPQC\Documents\CONDESA>
```

ILOB.2

main.cpp

```cpp
#include<iostream>
#include"stackList.h"

int main(){

    std :: cout << " --- PUSHING 'c' 'p' 'e' ONTO THE STACK ---- \n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); // Top should be 'e'
    displayALL<char>();
    std :: cout << "\n----- Poppping all the elements ----- \n";

    while(!isEmpty<char>()){
    std :: cout << " Popped: " << pop<char>() << std :: endl;}

    std :: cout << "\n ----- Testing the Underflow ----- \n";
    pop<char>();

    return 0;
}
```

Stacklist.h

```cpp
#ifndef STACK_LISI_H
#define STACK_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;     // VALUE THAT WE NEED STORE
    Node* next; // POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

// push
template <typename T>
void push(T newData) {
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data= newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}
// pop
template<typename T>
T pop(){
    //check if the stack is empty:
```

```
36
37      //save the current top node
38          Node<T>* temp = head<T>;
39      //store its data
40      T tempVal = temp->data;
41
42      // move the head to the next node
43      head<T> = head<T>->next;
44      //free the removed node
45      delete temp;
46      //return the removed value
47      return tempVal;
48  }
49
50  // isEmpty
51  template<typename T>
52  bool isEmpty(){
53      return head <T> == nullptr;
54  }
55
56  // top
57  template<typename T>
58  void Top(){
59      // error catching:
60      if(isEmpty<T>()){
61          std :: cout << "Stack underflow " << std :: endl;
62          return;
63      }
64      // display the Top node
```

```
62          return;
63      }
64      // display the Top node
65      std :: cout << "Top of Stack: " << head<T>->data << std :: endl;
66  }
67
68  //display All
69  template <typename T>
70  void displayALL(){
71      if (head<T> == nullptr){
72          std :: cout << "stack undeflow\n";
73          return;
74      }
75      std :: cout << "Displaying the stack: \n";
76      Node<T>* temp = head<T>;
77      while(temp != nullptr){
78          std :: cout << temp->data << "\n";
79          temp = temp->next;
80      }
81      std::cout << "\n";
82  }
83  #endif
84
```

COMPUTER ENGINEERING

QUEZON CITY

Supplementary Activity

Expression	Valid? (Y/N)	Output	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D]}			

Conclusion

Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

COMPUTER ENGINEERING

QUEZON CITY

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Overall, the activity showed that polymorphism makes programs more flexible by allowing code reuse, reducing repetition, and making it easier to maintain and improve.